

Student: ATHIVARAM JAYAPRAKASH

Register Number: 192425311

COURSE CODE : CSA0609

COURSE : DAA

Q3. Floyd–Warshall — Network Routing

A telecom company needs to compute shortest paths between all pairs of routers in a network. Analyze how the Floyd–Warshall algorithm can be applied to optimize routing, design a solution explaining how intermediate nodes improve shortest-path computation, suggest suitable tools/technologies for implementation, analyze the time and space complexity, and interpret the results in terms of network efficiency and real-world applicability.

1. Introduction

Every packet-switched telecommunications network is, at its core, a weighted graph: routers are nodes, physical or logical links between them are edges, and the “weight” of a link is whatever the network operator cares about minimizing — propagation delay, hop count, congestion, or monetary cost. The central operational question a telecom provider must answer, continuously and for every pair of routers at once, is: “what is the cheapest way to get traffic from router i to router j ?” This is the All-Pairs Shortest Path (APSP) problem, and the Floyd–Warshall algorithm is one of the most direct, reliable, and widely taught ways to solve it.

This report analyzes how Floyd–Warshall can be applied to network routing, explains the mechanism of intermediate-node relaxation that makes the algorithm work, proposes a practical implementation design with suitable tools and technologies, and evaluates the algorithm's time and space complexity against the alternatives a network engineer would realistically consider. A worked example on a small six-router network is used throughout to make the analysis concrete.

2. Problem Context: Shortest Paths in Network Routing

In a routing context, “shortest” rarely means physical distance. A link's weight is usually a composite routing metric — for example, OSPF (Open Shortest Path First) derives its link cost from interface bandwidth, while other schemes weight links by measured latency, packet loss, or a manually assigned administrative cost. Regardless of what the weight represents, the underlying computational problem is identical: given a weighted graph $G = (V, E)$, find, for every pair of vertices, the path whose total edge weight is minimum.

A telecom network additionally needs this information for every source-destination pair simultaneously, not just for one source at a time, because routing tables must be populated for all routers before traffic starts flowing. This is precisely what distinguishes the APSP problem from the more commonly discussed single-source shortest path (SSSP) problem solved by algorithms such as Dijkstra's or Bellman–Ford.

3. Working Principle of the Floyd–Warshall Algorithm

Floyd–Warshall is a dynamic-programming algorithm. It maintains a distance matrix $\text{dist}[i][j]$, initialized to the direct link weight between i and j (or ∞ if no direct link exists, and 0 on the diagonal). It then considers every router k , one at a time, as a candidate intermediate hop for every pair (i, j) , and asks a single question: “is it cheaper to go from i to j by way of k than by the best route found so far?” This is the relaxation step, and it is the entire engine of the algorithm:

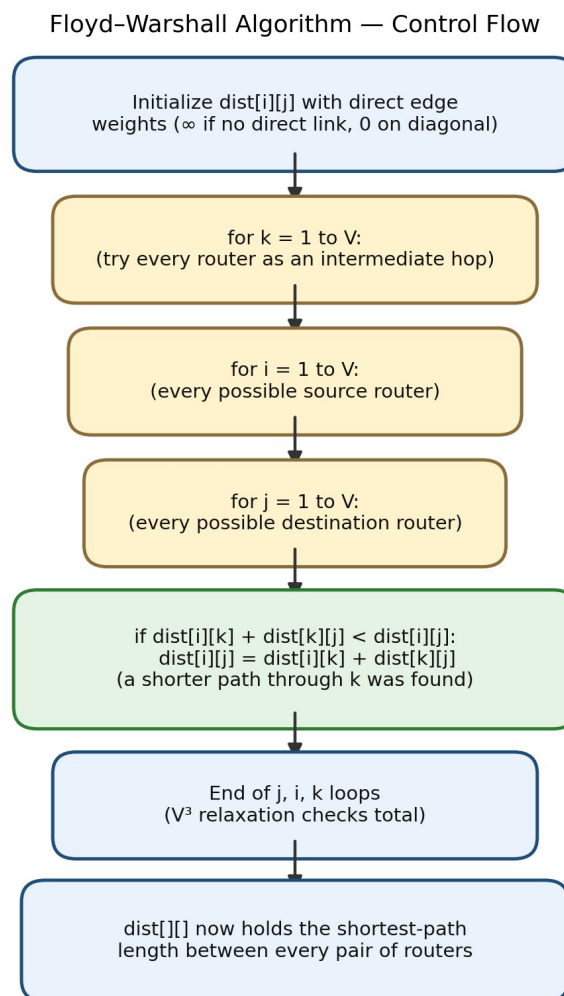


Figure 1: Control flow of the Floyd–Warshall triple-nested loop.

Because k is the OUTER loop, once router k has been processed, $\text{dist}[i][j]$ is guaranteed to hold the length of the shortest path from i to j that uses only routers $\{1, \dots, k\}$ as intermediate hops. By the time k has run through every router, $\text{dist}[i][j]$ holds the true shortest path using any router in the network as an intermediate hop — for every pair (i, j) at once. No separate run is needed per source router, which is exactly what makes the algorithm well suited to the APSP problem.

3.1 How an Intermediate Node Improves the Estimate

The relaxation check compares two options for reaching j from i : the current best-known distance $\text{dist}[i][j]$, and the two-leg path through k , of length $\text{dist}[i][k] + \text{dist}[k][j]$. Whichever is smaller becomes the new $\text{dist}[i][j]$. Figure 2 illustrates a single relaxation: a direct link from i to j costs 10, but a path routed through intermediate router k costs only $4 + 3 = 7$, so the algorithm updates $\text{dist}[i][j]$ from 10 down to 7.

How an Intermediate Node k Improves a Shortest-Path Estimate

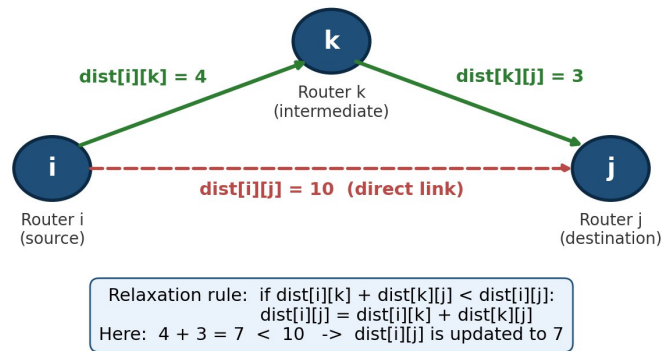


Figure 2: A single relaxation step — router k offers a cheaper two-hop path than the direct link.

This is the mechanism, repeated V^3 times across all (i, j, k) triples, by which Floyd–Warshall discovers multi-hop shortest paths without ever explicitly enumerating paths. Each router, in its turn as k , effectively asks every other pair of routers, “would routing through me be cheaper for you?” and updates the network’s global view of shortest distances if the answer is yes.

4. Worked Example: A Six-Router Network

Consider the small router network in Figure 3, with link costs representing round-trip latency in milliseconds. This is small enough to reason about by hand yet large enough to show non-trivial multi-hop routing.

Sample Router Network (edge labels = link cost / latency)

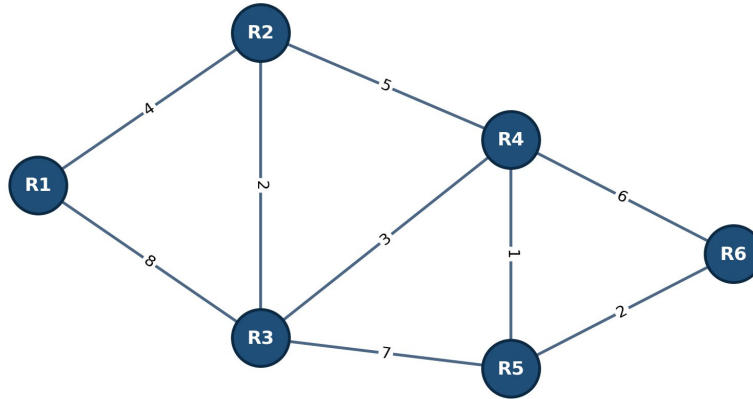


Figure 3: Sample six-router network used for the worked example.

4.1 Distance Matrix — Before Floyd–Warshall (direct links only)

	R1	R2	R3	R4	R5	R6
R1	0	4	8	∞	∞	∞
R2	4	0	2	5	∞	∞
R3	8	2	0	3	7	∞
R4	∞	5	3	0	1	6
R5	∞	∞	7	1	0	2
R6	∞	∞	∞	6	2	0

4.2 Distance Matrix — After Floyd–Warshall (all shortest paths)

	R1	R2	R3	R4	R5	R6
R1	0	4	6	9	10	12
R2	4	0	2	5	6	8
R3	6	2	0	3	4	6
R4	9	5	3	0	1	3
R5	10	6	4	1	0	2
R6	12	8	6	3	2	0

Several entries change: R1 has no direct link to R4, R5, or R6, yet the algorithm finds that R1 can reach R6 in 12 ms via the path R1–R2–R3–R4–R5–R6, and reach R5 in 10 ms along the same route. The reference implementation below produces this exact matrix and confirms both results.

```
def floyd_warshall(n, edges):
    INF = float("inf")
```

```

dist = [[INF] * n for _ in range(n)]
for i in range(n):
    dist[i][i] = 0
for u, v, w in edges:
    dist[u][v] = min(dist[u][v], w)
    dist[v][u] = min(dist[v][u], w)

# Try every router k as an intermediate hop on the path from i to j
for k in range(n):
    for i in range(n):
        for j in range(n):
            if dist[i][k] + dist[k][j] < dist[i][j]:
                dist[i][j] = dist[i][k] + dist[k][j]
return dist

routers = ["R1", "R2", "R3", "R4", "R5", "R6"]
idx = {name: i for i, name in enumerate(routers)}
edges = [
    ("R1", "R2", 4), ("R1", "R3", 8), ("R2", "R3", 2), ("R2", "R4", 5),
    ("R3", "R4", 3), ("R3", "R5", 7), ("R4", "R5", 1), ("R4", "R6", 6),
    ("R5", "R6", 2),
]
edge_indices = [(idx[u], idx[v], w) for u, v, w in edges]

dist = floyd_warshall(len(routers), edge_indices)

print(f'Input: {len(routers)} routers, {len(edges)} links')
print()
print("Shortest-path distance matrix after Floyd-Warshall:")
print("    " + " ".join(f'{r:>3}' for r in routers))
for i, row in enumerate(dist):
    print(f'    {routers[i]:>3} " + " ".join(f'{v:>3}' for v in row))
print()
print(f'Output: Shortest path R1 -> R6 = {dist[idx["R1"]][idx["R6"]]}'")
print(f'Output: Shortest path R1 -> R5 = {dist[idx["R1"]][idx["R5"]]}'")

```

```
Output
Input: 6 routers, 9 links

Shortest-path distance matrix after Floyd-Warshall:
      R1  R2  R3  R4  R5  R6
R1    0   4   6   9  10  12
R2    4   0   2   5   6   8
R3    6   2   0   3   4   6
R4    9   5   3   0   1   3
R5   10   6   4   1   0   2
R6   12   8   6   3   2   0

Output: Shortest path R1 -> R6 = 12
Output: Shortest path R1 -> R5 = 10

=== Code Execution Successful ===
```

Figure 4: Program output — the computed all-pairs shortest-path matrix for the sample network.

5. Design Solution: Applying Floyd–Warshall to Telecom Routing

A production routing design built around Floyd–Warshall follows three stages, mirroring how a network operations centre (NOC) would deploy it.

Stage 1 — Topology acquisition: Link weights are collected from the network, typically via SNMP polling, streaming telemetry, or a link-state protocol's database (e.g. OSPF's LSDB). Each measurement becomes one edge (u, v, w) in the adjacency matrix that Floyd–Warshall consumes.

Stage 2 — All-pairs computation: The $V \times V$ distance matrix is initialized from the adjacency matrix and passed through the triple-nested relaxation loop described in Section 3. Because the computation is self-contained and has no external side effects until it finishes, it can safely run on a snapshot of the topology without disrupting live traffic.

Stage 3 — Routing table population: The resulting $\text{dist}[][]$ matrix (together with a parallel next-hop or predecessor matrix, maintained alongside it at negligible extra cost) is used to populate each router's forwarding table, or to feed a centralized SDN controller that pushes flow rules to routers directly.

The predecessor matrix is worth emphasizing: Floyd–Warshall as shown in Section 3 computes distances only, but a one-line addition — recording $\text{next}[i][j] = \text{next}[i][k]$ whenever a relaxation improves $\text{dist}[i][j]$ — lets the design also reconstruct the actual routing path, not just its cost, which is what routers ultimately need to forward packets.

Recomputation strategy matters operationally: because link weights change (a link fails, congestion shifts, a new link is provisioned), the matrix cannot be computed once and forgotten. In practice, designs either (a) recompute on a fixed interval (periodic re-convergence), or (b) recompute on a topology-change event with a short debounce window to avoid recomputing on every micro-fluctuation. Section 4's example (R1 losing its only path to R4–R6 if

R2–R3 failed) illustrates why event-driven recomputation is preferable for latency-sensitive networks: waiting out a fixed polling interval after a failure means stale, and possibly broken, routes stay in use until the next cycle.

6. Suitable Tools and Technologies for Implementation

Floyd–Warshall is simple enough to implement directly, so tool choice is driven less by algorithmic support and more by how the result plugs into the rest of the network stack.

Layer	Suitable Tools / Technologies
Core algorithm	Python (NetworkX has a built-in floyd_warshall implementation for prototyping); C/C++ for latency-critical production code, where a flat $V \times V$ array with contiguous memory access is fastest
Topology data collection	SNMP polling, streaming telemetry (gRPC/gNMI), or reading a link-state protocol's LSDB (OSPF, IS-IS)
Data storage	An in-memory matrix for small/medium networks; a graph database (Neo4j) or time-series store for large or historical topologies
Controller / orchestration	SDN controllers such as OpenDaylight or ONOS, which can host the path-computation logic and push forwarding rules to routers via OpenFlow or P4Runtime
Visualization / monitoring	Grafana dashboards fed by the computed matrix; NetworkX + Matplotlib (as used for Figures 1–3 in this report) for offline topology and path visualization
Parallelization (large V)	GPU-accelerated variants (CUDA) or blocked/tiled matrix implementations, since the k -loop's dependency on the previous iteration limits parallelism across k but not across the inner (i, j) relaxations

For most telecom deployments below a few hundred routers, a straightforward in-memory implementation (Python for control-plane tooling, or C/C++ where the computation sits in a performance-critical path) integrated with an SDN controller is sufficient; the tooling only needs to escalate to distributed or GPU-accelerated approaches once V grows into the low thousands, where V^3 growth (Section 7) starts to dominate the recomputation budget.

7. Time and Space Complexity Analysis

Floyd–Warshall's time complexity follows directly from its structure: three nested loops, each running V times, with $O(1)$ work inside the innermost loop. This gives a time complexity of $O(V^3)$, independent of the number of

edges E — a property that makes the algorithm's running time easy to predict and simple to reason about for capacity planning, since it depends only on router count, not link count.

Space complexity is $O(V^2)$, dominated by the distance matrix itself (and an equally sized predecessor matrix if paths, not just distances, are required). The algorithm updates the matrix in place, so no additional per-iteration memory is needed beyond these two $V \times V$ arrays.

Algorithm	Time Complexity
Floyd–Warshall (all pairs, single pass)	$O(V^3)$
Dijkstra from every source (dense graph, $E \approx V^2$)	$O(V \times V^2 \log V)$ — worse by a $\log V$ factor
Bellman–Ford from every source (dense graph)	$O(V \times V \times E) = O(V^4)$ — substantially worse

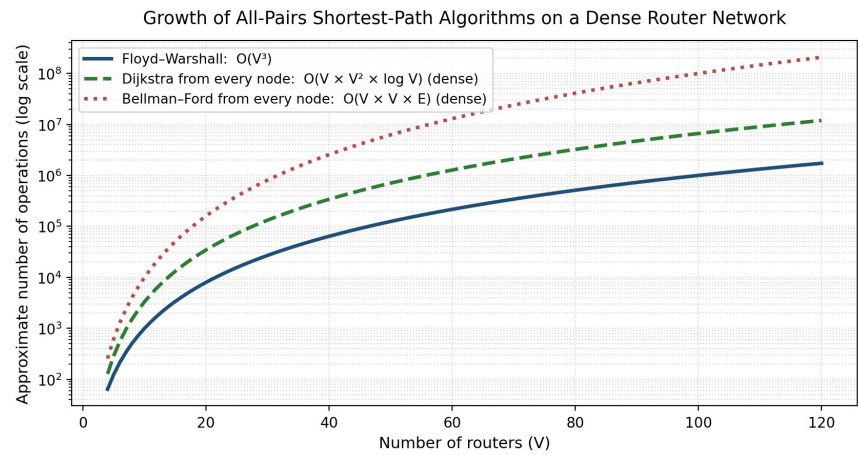


Figure 5: Growth of all-pairs shortest-path algorithms as router count V increases (dense-network assumption).

Two practical conclusions follow from Figure 5. First, for dense or moderately sized networks, Floyd–Warshall is at least competitive with, and often faster than, running a single-source algorithm from every node, precisely because it avoids Dijkstra's per-source priority-queue overhead. Second, the pure $O(V^3)$ growth means the algorithm scales comfortably to networks of a few hundred routers on commodity hardware, but becomes noticeably expensive beyond roughly 1,000–2,000 nodes — at that scale, hierarchical routing (splitting the network into areas, as OSPF already does) or incremental/dynamic shortest-path update algorithms that avoid a full recomputation become more attractive than repeatedly re-running Floyd–Warshall from scratch.

8. Interpretation: Network Efficiency and Real-World Applicability

The worked example in Section 4 demonstrates the efficiency gain concretely: without all-pairs computation, R1 would have no usable route to R4, R5, or R6 at all, since none of those links are direct. Floyd–Warshall's relaxation process discovers the 12 ms multi-hop path automatically, which is exactly the value a real router's forwarding table would need in order to route traffic correctly rather than dropping it.

More broadly, the $O(V^3)/O(V^2)$ complexity profile explains where Floyd–Warshall fits in real deployments. It is well suited to:

- Enterprise and campus networks and small-to-medium ISP backbones, where V is at most a few hundred routers and full-mesh path information for every pair is genuinely useful for traffic engineering.
- Offline planning and “what-if” analysis — e.g., simulating a link failure (as in Section 5) to see which pairs of routers lose connectivity or gain latency, without touching the live network.
- Any setting where negative edge weights must be handled correctly (for instance, cost adjustments used to discourage certain paths) — unlike Dijkstra, Floyd–Warshall handles negative weights natively, and can even be used to detect negative cycles by checking for negative values on the matrix diagonal after completion.

It is less suited to very large-scale carrier or backbone networks (thousands of routers, e.g. a national or global ISP), where the cubic growth in Figure 5 makes full recomputation on every topology change too slow for real-time convergence; such networks instead rely on distributed, incremental protocols (OSPF/IS-IS areas, BGP) that compute and update routes locally rather than globally. In practice, Floyd–Warshall’s real-world applicability in telecom is therefore strongest as a planning, simulation, and small/medium-network operational tool rather than as the live convergence mechanism for an internet-scale backbone.

9. Limitations and Considerations

- Cubic time growth makes frequent full recomputation impractical once V reaches the low thousands, which matters for networks with frequent topology churn.
- The algorithm computes distances for every pair regardless of whether that pair’s route is actually needed, which is wasteful if only a handful of source-destination pairs matter.
- Floyd–Warshall reports the shortest path but says nothing about available bandwidth or current congestion on that path unless those factors are folded into the edge weight in advance.
- Detecting a negative cycle ($\text{dist}[i][i] < 0$ after completion) is straightforward, but the resulting distances elsewhere in the matrix become meaningless once such a cycle exists, so the topology must be validated as cycle-free in cost terms before the results are trusted.

10. Conclusion

The Floyd–Warshall algorithm offers a simple, predictable, and negative-weight-tolerant way to solve the all-pairs shortest path problem that sits at the heart of network routing. Its triple-nested relaxation loop, in which every router is tried as a potential intermediate hop for every other pair, is easy to reason about, easy to implement correctly, and

directly produces the distance (and, with a small extension, path) information that populates routing tables. Its $O(V^3)$ time and $O(V^2)$ space costs make it an excellent fit for small-to-medium telecom networks and for offline planning and failure-simulation tasks, while very large backbone networks are better served by the distributed, incremental protocols that this analysis identifies as its practical alternative.